10 Python Programming Problems

Problem Statements • Algorithms • Flowcharts • Source Code • Sample Input & Output

Beginner-friendly practical record for first-year engineering students

Note: Flowcharts are included as diagrams. The dark output panels are sample console-output screenshots recreated inside the PDF; they are examples, not screenshots captured from a live computer.

1. University Scholarship Allocation

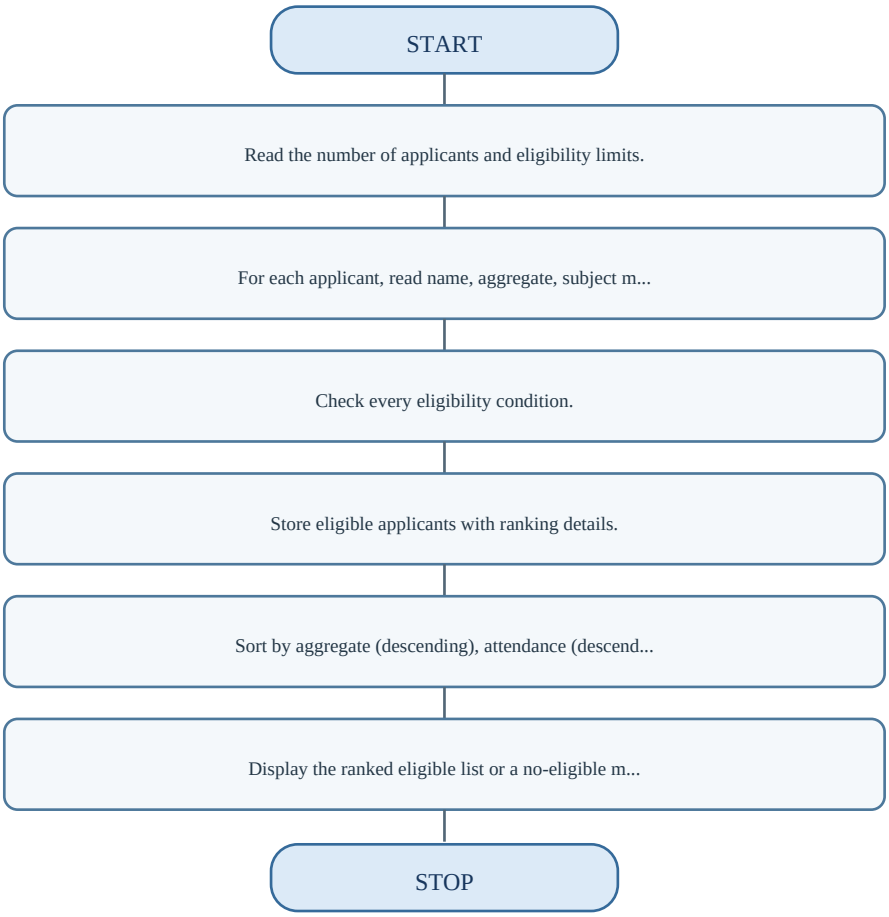
Problem Statement

Create a system that checks applicants using aggregate marks, minimum subject marks, attendance and family income. Eligible applicants are ranked by aggregate marks; ties are resolved by higher attendance, then lower family income.

Algorithm

1. Read the number of applicants and eligibility limits.
2. For each applicant, read name, aggregate, subject marks, attendance and annual income.
3. Check every eligibility condition.
4. Store eligible applicants with ranking details.
5. Sort by aggregate (descending), attendance (descending), and income (ascending).
6. Display the ranked eligible list or a no-eligible message.

Flowchart



Python Source Code

```
n = int(input("Number of applicants: "))
min_aggregate = float(input("Minimum aggregate: "))
min_subject = float(input("Minimum subject mark: "))
min_attendance = float(input("Minimum attendance: "))
max_income = float(input("Maximum family income: "))
```

```
eligible = []
for i in range(n):
    print("\\nApplicant", i + 1)
    name = input("Name: ")
    aggregate = float(input("Aggregate marks: "))
    s1 = float(input("Subject 1 marks: "))
    s2 = float(input("Subject 2 marks: "))
    attendance = float(input("Attendance percentage: "))
    income = float(input("Annual family income: "))

    if (aggregate >= min_aggregate and s1 >= min_subject
        and s2 >= min_subject and attendance >= min_attendance
        and income <= max_income):
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("\\n--- SCHOLARSHIP RANK LIST ---")
if not eligible:
    print("No eligible applicants")
else:
    for rank, student in enumerate(eligible, 1):
        print(rank, student[0], "Marks:", student[1],
              "Attendance:", student[2], "Income:", student[3])
```

Sample Input

```
Number of applicants: 2
Minimum aggregate: 80
Minimum subject mark: 70
Minimum attendance: 75
Maximum family income: 200000

Applicant 1
Name: Anu
Aggregate marks: 88
Subject 1 marks: 85
Subject 2 marks: 80
Attendance percentage: 90
Annual family income: 150000

Applicant 2
Name: Ravi
Aggregate marks: 82
Subject 1 marks: 75
Subject 2 marks: 78
Attendance percentage: 85
Annual family income: 180000
```

Sample Output – Console Screenshot

```
--- SCHOLARSHIP RANK LIST ---
1 Anu Marks: 88.0 Attendance: 90.0 Income: 150000.0
2 Ravi Marks: 82.0 Attendance: 85.0 Income: 180000.0
```

Result: The program performs the required calculations and displays the result according to the stated rules.

2. Progressive Electricity Billing

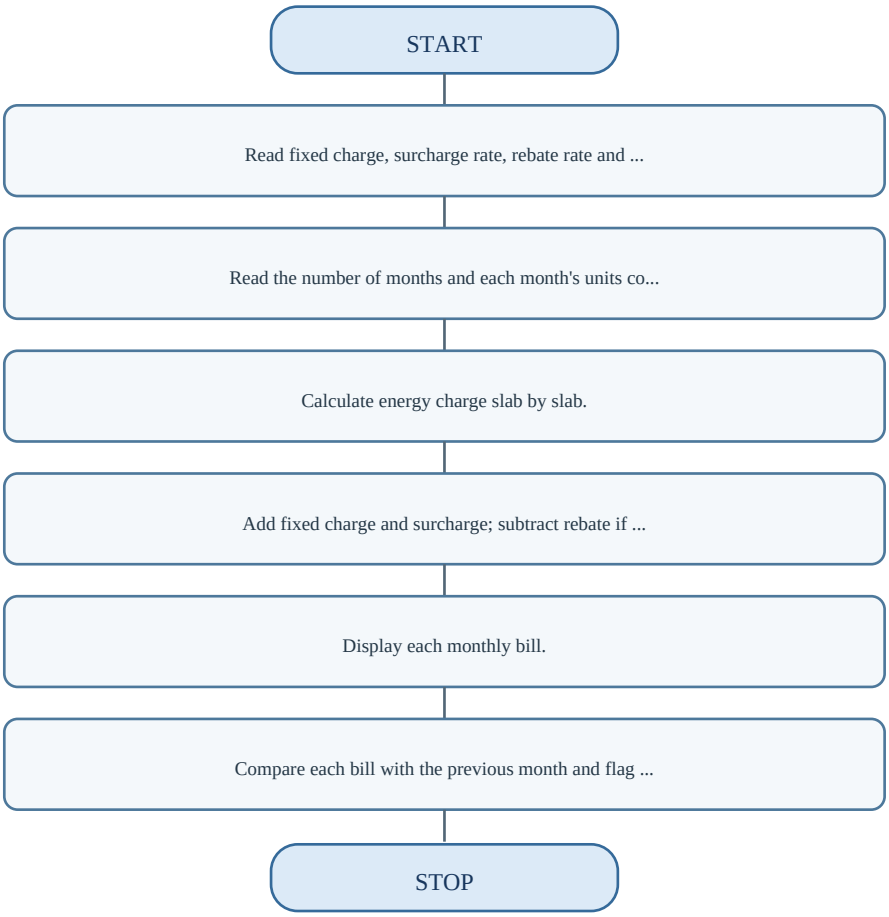
Problem Statement

Calculate monthly electricity bills using consumption slabs, fixed charges, surcharge and rebate. Compare consecutive monthly bills and flag an unusual increase when the bill rises by more than the supplied percentage.

Algorithm

1. Read fixed charge, surcharge rate, rebate rate and unusual-increase limit.
2. Read the number of months and each month's units consumed.
3. Calculate energy charge slab by slab.
4. Add fixed charge and surcharge; subtract rebate if applicable.
5. Display each monthly bill.
6. Compare each bill with the previous month and flag unusual increases.

Flowchart



Python Source Code

```
fixed = float(input("Fixed charge: "))
surcharge_rate = float(input("Surcharge percentage: "))
rebate_rate = float(input("Rebate percentage: "))
limit = float(input("Unusual increase percentage: "))
n = int(input("Number of months: "))
```

```
bills = []
for i in range(n):
    units = float(input("Units for month " + str(i + 1) + ": "))
    # Slabs: first 100 at 1.5, next 100 at 2.5, above 200 at 4
    a = min(units, 100) * 1.5
    b = max(0, min(units - 100, 100)) * 2.5
    c = max(0, units - 200) * 4
    energy = a + b + c
    subtotal = energy + fixed
    surcharge = subtotal * surcharge_rate / 100
    rebate = energy * rebate_rate / 100
    bill = subtotal + surcharge - rebate
    bills.append(bill)
    print("Month", i + 1, "bill: Rs.", round(bill, 2))

print("\\n--- MONTHLY COMPARISON ---")
for i in range(1, n):
    increase = bills[i] - bills[i - 1]
    percent = increase / bills[i - 1] * 100 if bills[i - 1] else 0
    print("Month", i + 1, "increase:", round(percent, 2), "%")
    if percent > limit:
        print("Unusual increase detected")
```

Sample Input

```
Fixed charge: 50
Surcharge percentage: 5
Rebate percentage: 2
Unusual increase percentage: 30
Number of months: 3
Units for month 1: 80
Units for month 2: 120
Units for month 3: 260
```

Sample Output – Console Screenshot

```
Month 1 bill: Rs. 169.40
Month 2 bill: Rs. 226.80
Month 3 bill: Rs. 806.40

--- MONTHLY COMPARISON ---
Month 2 increase: 33.92 %
Unusual increase detected
Month 3 increase: 255.56 %
Unusual increase detected
```

Result: The program performs the required calculations and displays the result according to the stated rules.

3. Hospital Billing and Patient Classification

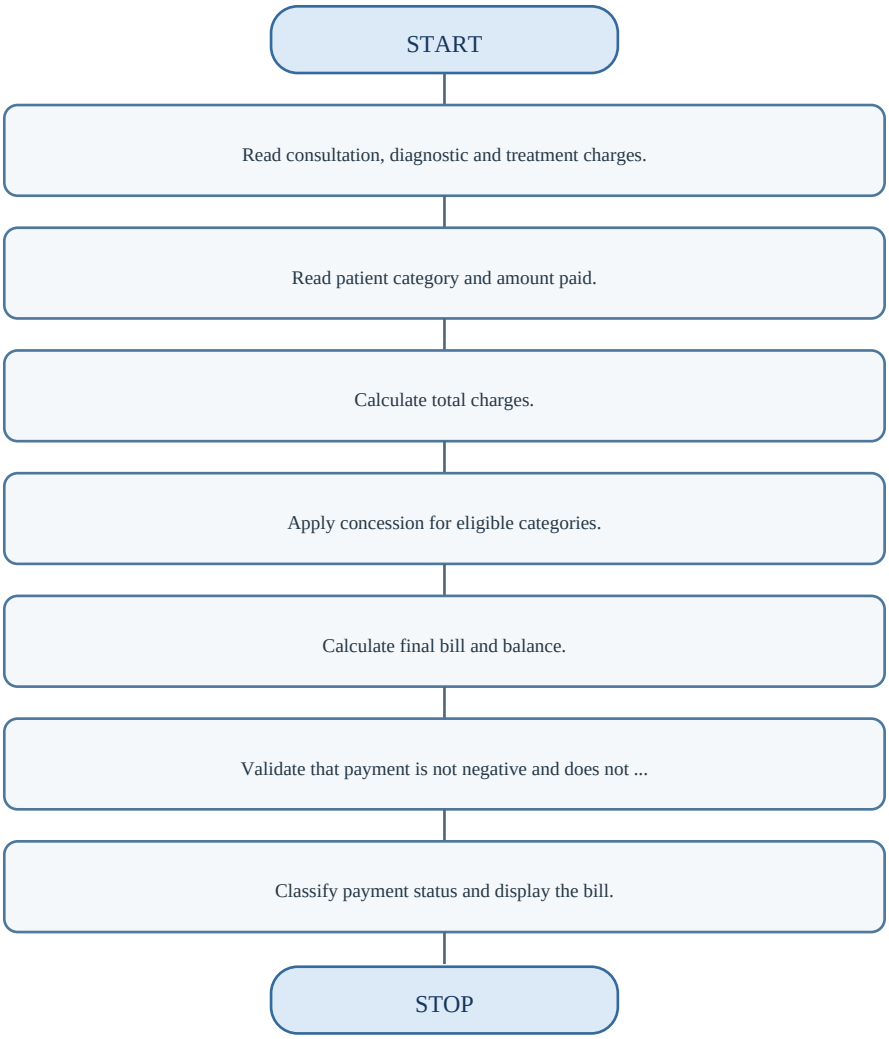
Problem Statement

Calculate consultation, diagnostic and treatment charges. Apply a concession according to patient type, validate the payment amount, and classify the payment status as Paid, Partially Paid or Pending.

Algorithm

1. Read consultation, diagnostic and treatment charges.
2. Read patient category and amount paid.
3. Calculate total charges.
4. Apply concession for eligible categories.
5. Calculate final bill and balance.
6. Validate that payment is not negative and does not exceed the final bill.
7. Classify payment status and display the bill.

Flowchart



Python Source Code

```
consult = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))
```

```
category = input("Patient type (senior/regular): ").lower()
paid = float(input("Amount paid: "))

total = consult + diagnostic + treatment
discount = total * 10 / 100 if category == "senior" else 0
final_bill = total - discount

if paid < 0 or paid > final_bill:
    print("Invalid payment amount")
else:
    balance = final_bill - paid
    if balance == 0:
        status = "Paid"
    elif paid > 0:
        status = "Partially Paid"
    else:
        status = "Pending"

    print("\\n--- HOSPITAL BILL ---")
    print("Total charges: Rs.", round(total, 2))
    print("Concession: Rs.", round(discount, 2))
    print("Final bill: Rs.", round(final_bill, 2))
    print("Paid: Rs.", round(paid, 2))
    print("Balance: Rs.", round(balance, 2))
    print("Patient payment status:", status)
```

Sample Input

```
Consultation charge: 500
Diagnostic charge: 1500
Treatment charge: 3000
Patient type (senior/regular): senior
Amount paid: 3000
```

Sample Output – Console Screenshot

```
--- HOSPITAL BILL ---
Total charges: Rs. 5000.0
Concession: Rs. 500.0
Final bill: Rs. 4500.0
Paid: Rs. 3000.0
Balance: Rs. 1500.0
Patient payment status: Partially Paid
```

Result: The program performs the required calculations and displays the result according to the stated rules.

4. Employee Payroll and Deduction Analysis

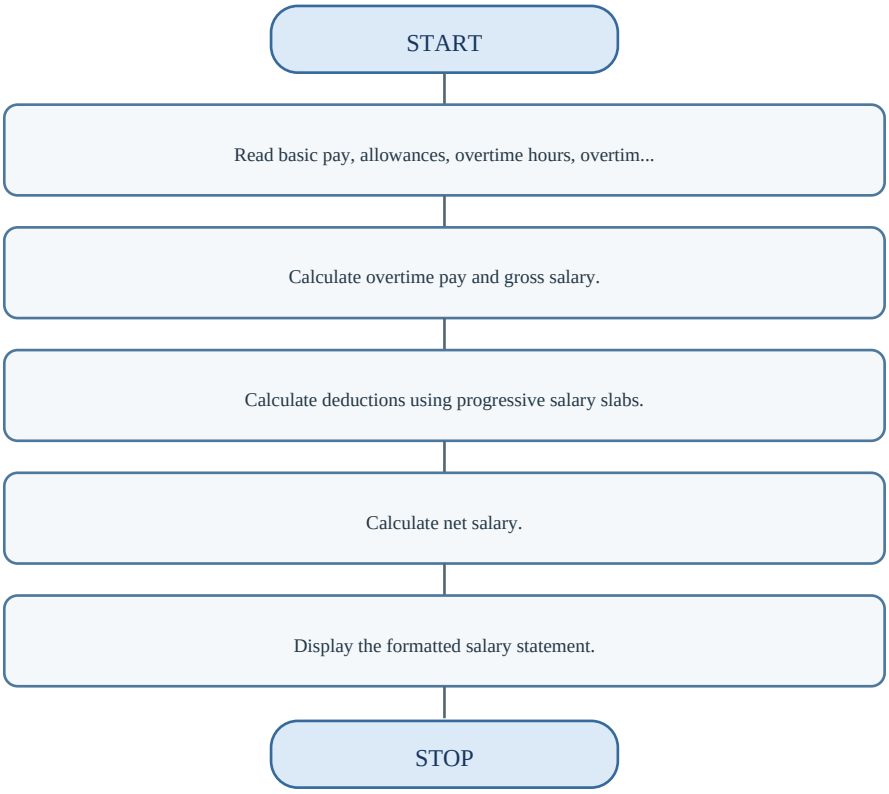
Problem Statement

Calculate gross salary from basic pay, allowances, overtime and incentives. Apply progressive deductions: 5% on the first Rs. 10,000, 10% on the next Rs. 10,000 and 15% above Rs. 20,000. Generate a salary statement.

Algorithm

1. Read basic pay, allowances, overtime hours, overtime rate and incentives.
2. Calculate overtime pay and gross salary.
3. Calculate deductions using progressive salary slabs.
4. Calculate net salary.
5. Display the formatted salary statement.

Flowchart



Python Source Code

```
basic = float(input("Basic pay: "))
allowances = float(input("Allowances: "))
hours = float(input("Overtime hours: "))
rate = float(input("Overtime rate per hour: "))
incentive = float(input("Incentives: "))

overtime = hours * rate
```

```
gross = basic + allowances + overtime + incentive

deduction = (min(gross, 10000) * 0.05
             + max(0, min(gross - 10000, 10000)) * 0.10
             + max(0, gross - 20000) * 0.15)
net = gross - deduction

print("\\n--- SALARY STATEMENT ---")
print("Basic pay: Rs.", basic)
print("Allowances: Rs.", allowances)
print("Overtime pay: Rs.", overtime)
print("Incentives: Rs.", incentive)
print("Gross salary: Rs.", round(gross, 2))
print("Total deductions: Rs.", round(deduction, 2))
print("Net salary: Rs.", round(net, 2))
```

Sample Input

```
Basic pay: 20000
Allowances: 5000
Overtime hours: 10
Overtime rate per hour: 100
Incentives: 2000
```

Sample Output – Console Screenshot

```
--- SALARY STATEMENT ---
Basic pay: Rs. 20000.0
Allowances: Rs. 5000.0
Overtime pay: Rs. 1000.0
Incentives: Rs. 2000.0
Gross salary: Rs. 28000.0
Total deductions: Rs. 3200.0
Net salary: Rs. 24800.0
```

Result: The program performs the required calculations and displays the result according to the stated rules.

5. Multi-Plan Telecommunications Cost Optimization

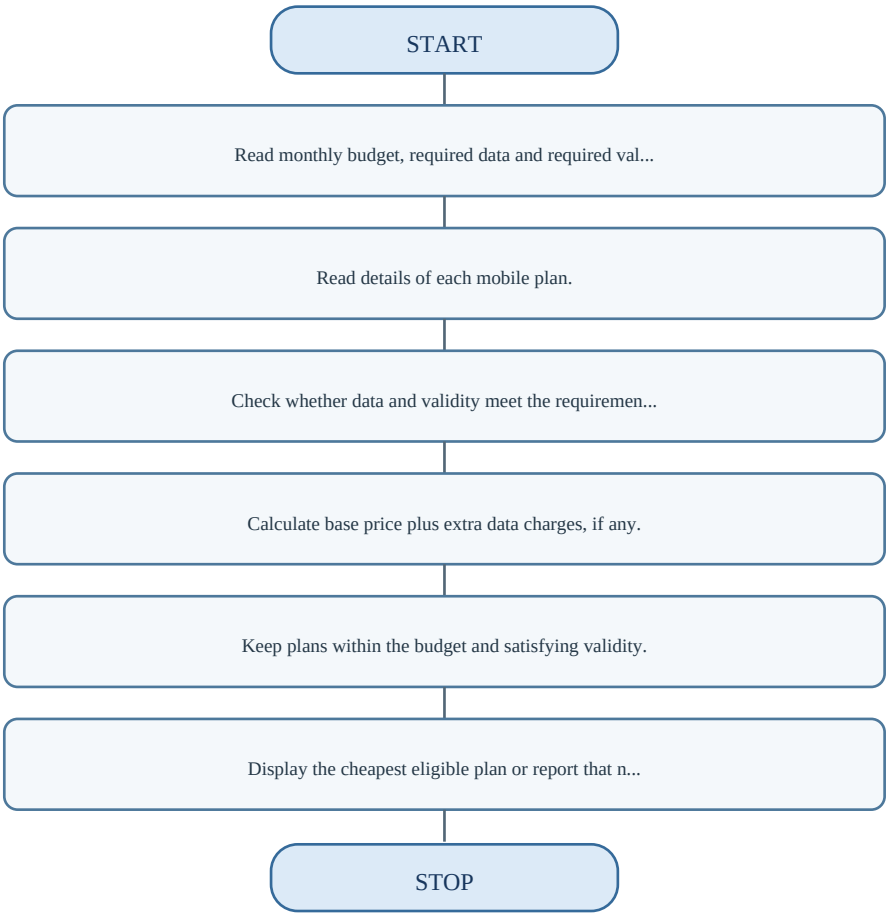
Problem Statement

Compare mobile plans based on monthly budget, required data and validity. Calculate extra data charges and select the lowest-cost plan that satisfies the user's requirements.

Algorithm

1. Read monthly budget, required data and required validity.
2. Read details of each mobile plan.
3. Check whether data and validity meet the requirements.
4. Calculate base price plus extra data charges, if any.
5. Keep plans within the budget and satisfying validity.
6. Display the cheapest eligible plan or report that none is suitable.

Flowchart



Python Source Code

```
budget = float(input("Monthly budget: "))
need_data = float(input("Required data in GB: "))
need_days = int(input("Required validity in days: "))
n = int(input("Number of plans: "))
```

```
best_name = ""
best_cost = float("inf")

for i in range(n):
    print("\\nPlan", i + 1)
    name = input("Plan name: ")
    price = float(input("Plan price: "))
    data = float(input("Included data in GB: "))
    days = int(input("Validity days: "))
    extra_rate = float(input("Extra charge per GB: "))

    extra = max(0, need_data - data) * extra_rate
    cost = price + extra

    if days >= need_days and cost <= budget and cost < best_cost:
        best_name = name
        best_cost = cost

if best_name:
    print("\\nBest eligible plan:", best_name)
    print("Total cost: Rs.", round(best_cost, 2))
else:
    print("No plan meets all requirements and budget")
```

Sample Input

```
Monthly budget: 500
Required data in GB: 2
Required validity in days: 28
Number of plans: 2

Plan 1
Plan name: Basic
Plan price: 199
Included data in GB: 1.5
Validity days: 28
Extra charge per GB: 50

Plan 2
Plan name: Smart
Plan price: 249
Included data in GB: 2.5
Validity days: 28
Extra charge per GB: 50
```

Sample Output – Console Screenshot

```
Best eligible plan: Smart
Total cost: Rs. 249.0
```

Result: The program performs the required calculations and displays the result according to the stated rules.

6. Railway Fare and Reservation Calculator

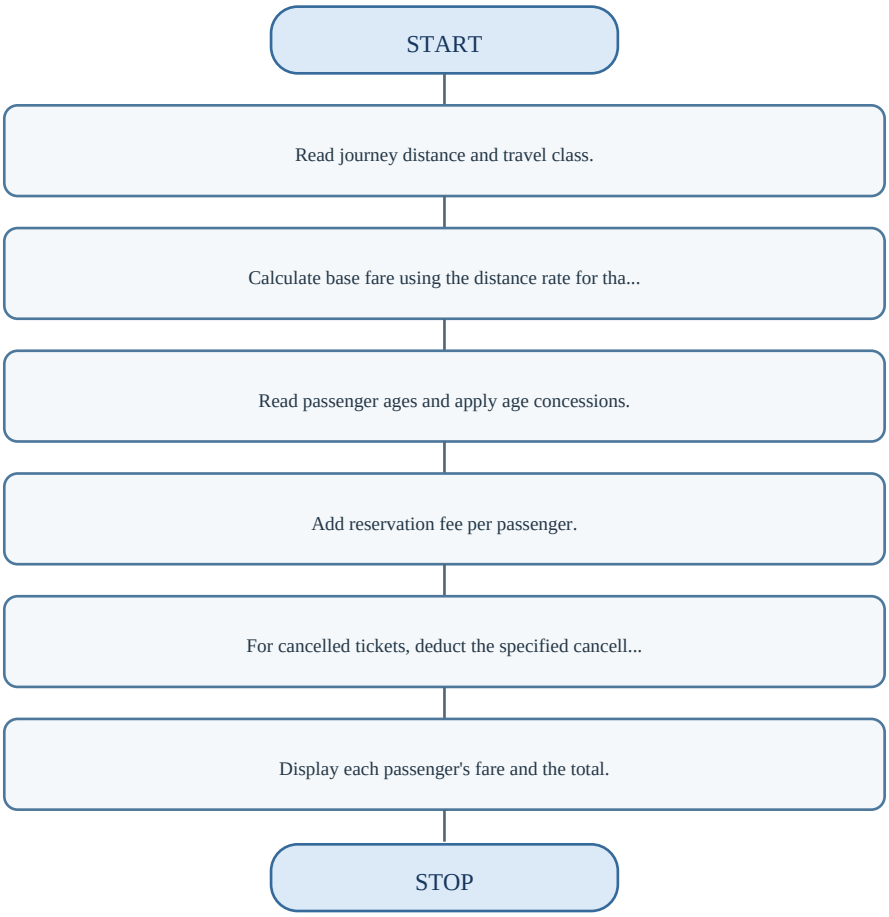
Problem Statement

Calculate fares for multiple passengers using distance slabs, travel class, age concessions and reservation fees. Apply a cancellation penalty when a ticket is cancelled.

Algorithm

1. Read journey distance and travel class.
2. Calculate base fare using the distance rate for that class.
3. Read passenger ages and apply age concessions.
4. Add reservation fee per passenger.
5. For cancelled tickets, deduct the specified cancellation penalty from the refundable fare.
6. Display each passenger's fare and the total.

Flowchart



Python Source Code

```
distance = float(input("Journey distance in km: "))
travel_class = input("Class (sleeper/ac/first): ").lower()
count = int(input("Number of passengers: "))

rates = {"sleeper": 1.5, "ac": 3.0, "first": 5.0}
```

```
if travel_class not in rates:
    print("Invalid class")
else:
    total = 0
    for i in range(count):
        age = int(input("Passenger age: "))
        cancelled = input("Cancelled? (yes/no): ").lower()
        fare = distance * rates[travel_class]

        if age < 5:
            fare = 0
        elif age >= 60:
            fare = fare * 0.8

        fare += 40 if fare > 0 else 0

        if cancelled == "yes":
            fare = max(0, fare - 100)

        print("Passenger", i + 1, "fare/refund: Rs.", round(fare, 2))
        total += fare

    print("Total amount: Rs.", round(total, 2))
```

Sample Input

```
Journey distance in km: 200
Class (sleeper/ac/first): sleeper
Number of passengers: 2
Passenger age: 30
Cancelled? (yes/no): no
Passenger age: 65
Cancelled? (yes/no): no
```

Sample Output – Console Screenshot

```
Passenger 1 fare/refund: Rs. 340.0
Passenger 2 fare/refund: Rs. 280.0
Total amount: Rs. 620.0
```

Result: The program performs the required calculations and displays the result according to the stated rules.

7. Retail Billing with Multiple Discount Rules

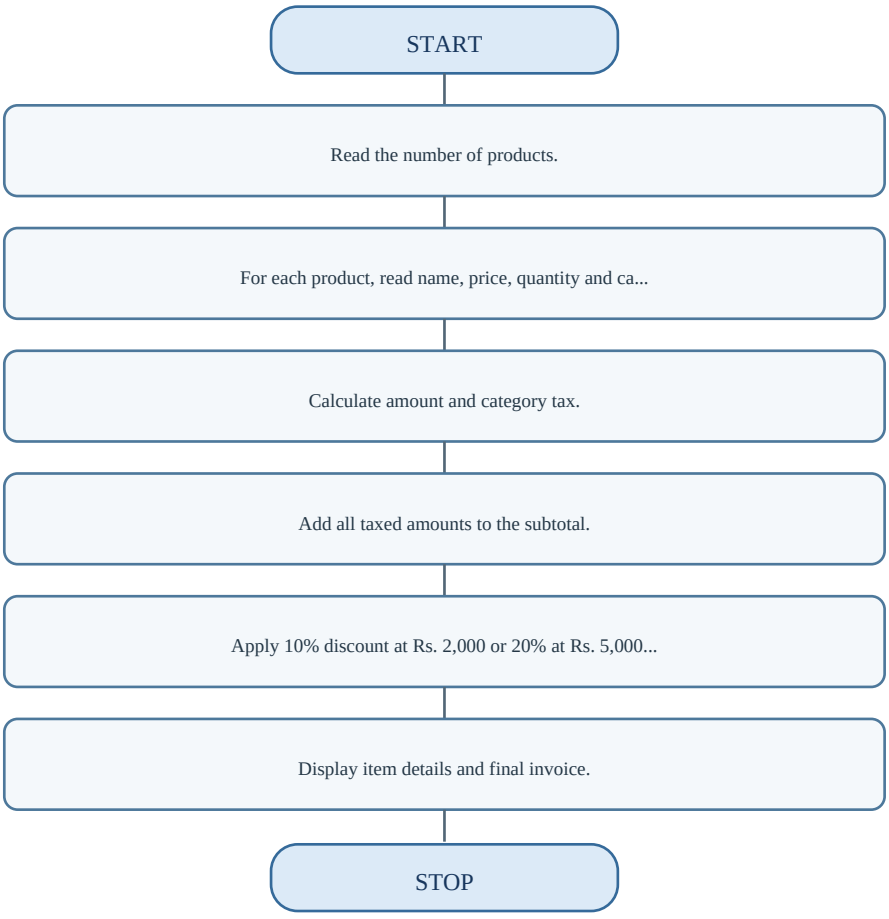
Problem Statement

Process multiple products, quantities, category-specific taxes, threshold discounts and a maximum discount limit. Generate an itemized invoice.

Algorithm

1. Read the number of products.
2. For each product, read name, price, quantity and category.
3. Calculate amount and category tax.
4. Add all taxed amounts to the subtotal.
5. Apply 10% discount at Rs. 2,000 or 20% at Rs. 5,000; cap discount at Rs. 1,000.
6. Display item details and final invoice.

Flowchart



Python Source Code

```
n = int(input("Number of products: "))
subtotal = 0
print("\\n--- ITEM DETAILS ---")

for i in range(n):
```

```
    name = input("Product name: ")
    price = float(input("Price: "))
    qty = int(input("Quantity: "))
    category = input("Category (food/clothing/electronics): ").lower()

    amount = price * qty
    if category == "food":
        tax_rate = 5
    elif category == "clothing":
        tax_rate = 5
    else:
        tax_rate = 18

    tax = amount * tax_rate / 100
    line_total = amount + tax
    subtotal += line_total
    print(name, "Amount:", amount, "Tax:", tax,
          "Line total:", line_total)

if subtotal >= 5000:
    discount = subtotal * 0.20
elif subtotal >= 2000:
    discount = subtotal * 0.10
else:
    discount = 0

discount = min(discount, 1000)
final_bill = subtotal - discount

print("\\nSubtotal:", round(subtotal, 2))
print("Discount:", round(discount, 2))
print("Final bill: Rs.", round(final_bill, 2))
```

Sample Input

```
Number of products: 2
Product name: Shirt
Price: 1000
Quantity: 2
Category (food/clothing/electronics): clothing
Product name: Mobile
Price: 5000
Quantity: 1
Category (food/clothing/electronics): electronics
```

Sample Output – Console Screenshot

```
--- ITEM DETAILS ---
Shirt Amount: 2000.0 Tax: 100.0 Line total: 2100.0
Mobile Amount: 5000.0 Tax: 900.0 Line total: 5900.0

Subtotal: 8000.0
Discount: 1000
Final bill: Rs. 7000.0
```

Result: The program performs the required calculations and displays the result according to the stated rules.

8. Manufacturing Cost and Profitability Analysis

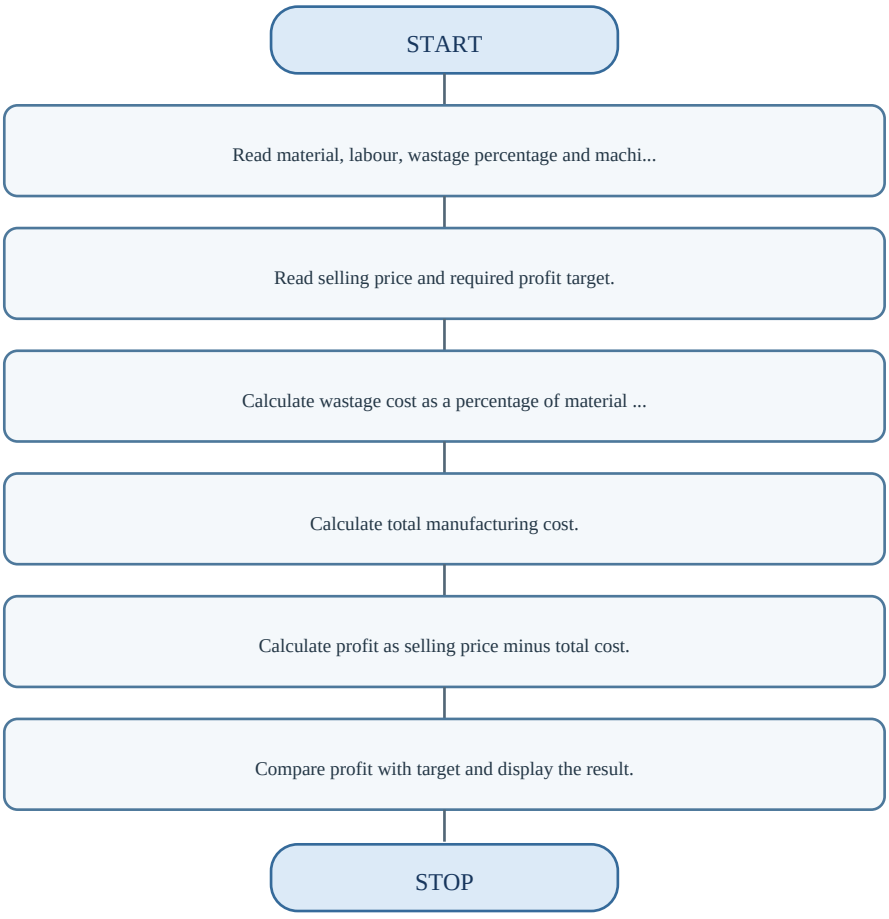
Problem Statement

Calculate material cost, labour charges, wastage cost and machine cost. Find total production cost and profit, then check whether the specified profit target is achieved.

Algorithm

1. Read material, labour, wastage percentage and machine costs.
2. Read selling price and required profit target.
3. Calculate wastage cost as a percentage of material cost.
4. Calculate total manufacturing cost.
5. Calculate profit as selling price minus total cost.
6. Compare profit with target and display the result.

Flowchart



Python Source Code

```
material = float(input("Material cost: "))
labour = float(input("Labour charges: "))
waste_rate = float(input("Wastage percentage: "))
machine = float(input("Machine cost: "))
selling = float(input("Selling price: "))
```

```
target = float(input("Profit target: "))

waste_cost = material * waste_rate / 100
total_cost = material + labour + waste_cost + machine
profit = selling - total_cost

print("\\n--- MANUFACTURING REPORT ---")
print("Wastage cost: Rs.", round(waste_cost, 2))
print("Total manufacturing cost: Rs.", round(total_cost, 2))
print("Selling price: Rs.", round(selling, 2))
print("Profit: Rs.", round(profit, 2))

if profit >= target:
    print("Profit target achieved")
else:
    print("Profit target not achieved")
```

Sample Input

```
Material cost: 5000
Labour charges: 2000
Wastage percentage: 10
Machine cost: 1000
Selling price: 10000
Profit target: 1000
```

Sample Output – Console Screenshot

Result: The program performs the required calculations and displays the result according to the stated rules.

9. Multi-Currency Travel Budget Analysis

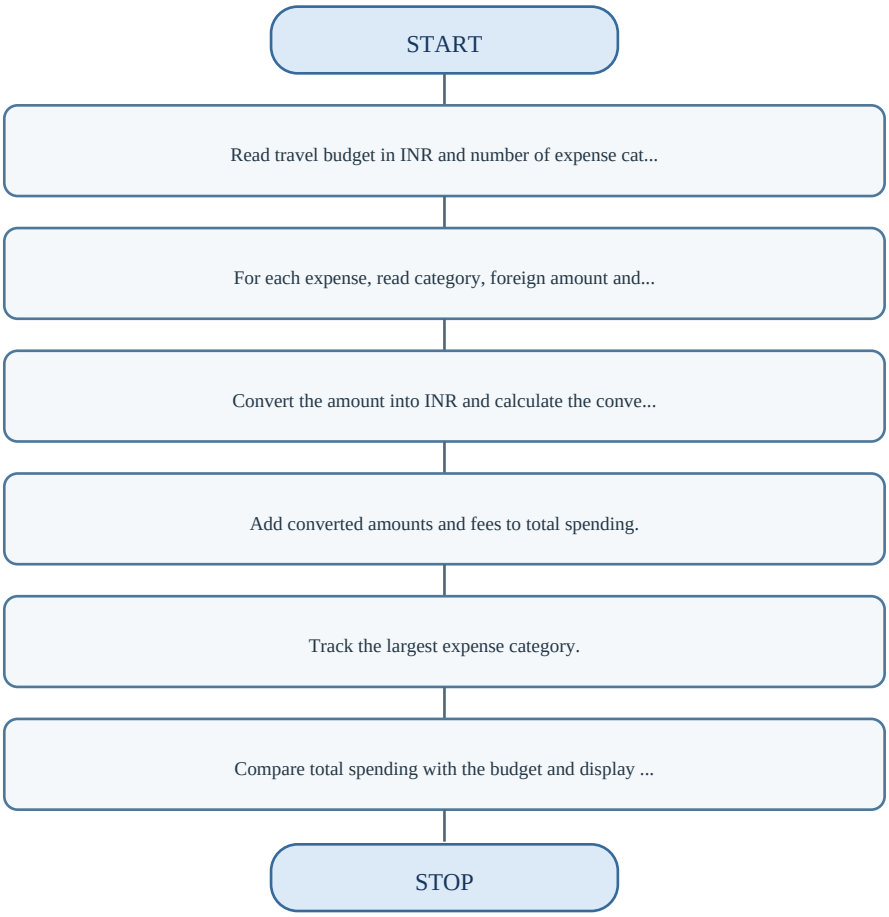
Problem Statement

Convert travel expenses into Indian Rupees using supplied exchange rates, add a conversion fee, compare total expenses with the budget and identify the largest expense category.

Algorithm

1. Read travel budget in INR and number of expense categories.
2. For each expense, read category, foreign amount and INR exchange rate.
3. Convert the amount into INR and calculate the conversion fee.
4. Add converted amounts and fees to total spending.
5. Track the largest expense category.
6. Compare total spending with the budget and display the report.

Flowchart



Python Source Code

```
budget = float(input("Travel budget in INR: "))
n = int(input("Number of expense categories: "))
fee_rate = float(input("Conversion fee percentage: "))

total = 0
```

```
largest = -1
largest_category = ""

for i in range(n):
    category = input("Expense category: ")
    amount = float(input("Amount in foreign currency: "))
    rate = float(input("Exchange rate (INR per unit): "))

    converted = amount * rate
    fee = converted * fee_rate / 100
    expense = converted + fee
    total += expense

    if expense > largest:
        largest = expense
        largest_category = category

    print("INR amount:", round(converted, 2),
          "Fee:", round(fee, 2),
          "Total:", round(expense, 2))

print("\\n--- TRAVEL BUDGET REPORT ---")
print("Total spending: Rs.", round(total, 2))
print("Largest category:", largest_category)
if total <= budget:
    print("Within budget")
else:
    print("Over budget")
print("Budget balance: Rs.", round(budget - total, 2))
```

Sample Input

```
Travel budget in INR: 50000
Number of expense categories: 3
Conversion fee percentage: 2
Expense category: Hotel
Amount in foreign currency: 100
Exchange rate (INR per unit): 85
Expense category: Food
Amount in foreign currency: 50
Exchange rate (INR per unit): 85
Expense category: Shopping
Amount in foreign currency: 200
Exchange rate (INR per unit): 85
```

Sample Output – Console Screenshot

```
INR amount: 8500.0 Fee: 170.0 Total: 8670.0
INR amount: 4250.0 Fee: 85.0 Total: 4335.0
INR amount: 17000.0 Fee: 340.0 Total: 17340.0

--- TRAVEL BUDGET REPORT ---
Total spending: Rs. 30345.0
Largest category: Shopping
Within budget
Budget balance: Rs. 19655.0
```

Result: The program performs the required calculations and displays the result according to the stated rules.

10. Campus Event Financial Planning

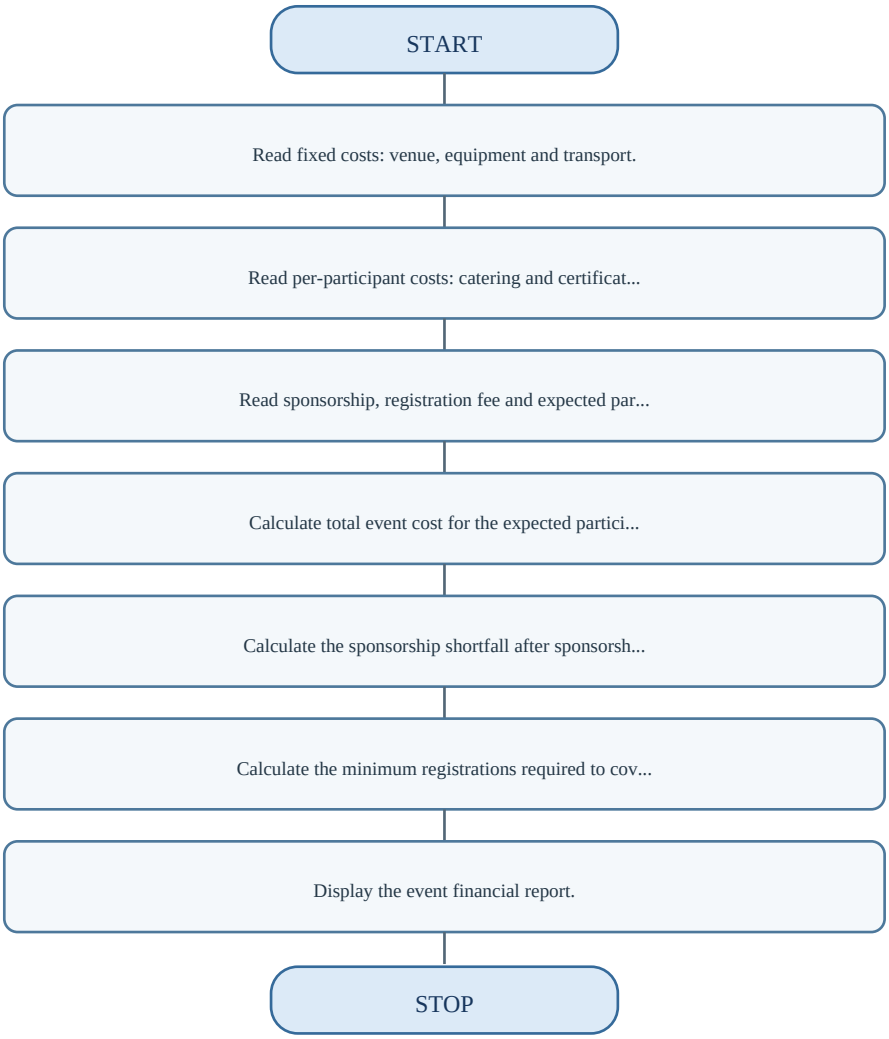
Problem Statement

Calculate event costs for venue, equipment, transport, catering and certificates. Find the participant contribution, sponsorship shortfall and minimum registrations needed to break even.

Algorithm

1. Read fixed costs: venue, equipment and transport.
2. Read per-participant costs: catering and certificates.
3. Read sponsorship, registration fee and expected participants.
4. Calculate total event cost for the expected participants.
5. Calculate the sponsorship shortfall after sponsorship and expected registration income.
6. Calculate the minimum registrations required to cover fixed costs after sponsorship, using contribution per registration.
7. Display the event financial report.

Flowchart



Python Source Code

```
venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
transport = float(input("Transport cost: "))
```

```
catering = float(input("Catering cost per participant: "))
certificate = float(input("Certificate cost per participant: "))
sponsorship = float(input("Sponsorship amount: "))
fee = float(input("Registration fee per participant: "))
participants = int(input("Expected participants: "))

fixed = venue + equipment + transport
variable = catering + certificate
total_cost = fixed + variable * participants
registration_income = fee * participants
funds = sponsorship + registration_income
shortfall = max(0, total_cost - funds)

if fee > variable:
    needed = max(0, math.ceil((fixed - sponsorship) / (fee - variable)))
else:
    needed = None

print("\\n--- EVENT FINANCIAL REPORT ---")
print("Fixed costs: Rs.", round(fixed, 2))
print("Variable cost per participant: Rs.", round(variable, 2))
print("Total event cost: Rs.", round(total_cost, 2))
print("Registration income: Rs.", round(registration_income, 2))
print("Sponsorship: Rs.", round(sponsorship, 2))
print("Funding shortfall: Rs.", round(shortfall, 2))
if needed is None:
    print("Break-even is not possible at this registration fee")
else:
    print("Minimum break-even registrations:", needed)
```

Sample Input

```
Venue cost: 10000
Equipment cost: 5000
Transport cost: 3000
Catering cost per participant: 200
Certificate cost per participant: 50
Sponsorship amount: 10000
Registration fee per participant: 400
Expected participants: 100
```

Sample Output – Console Screenshot

Result: The program performs the required calculations and displays the result according to the stated rules.